

SmartBugs - New Features Test Execution Report

Luciano Bercini

April 2026

Contents

1	Introduction	2
2	Test Environment	2
3	Executed Test Suites	2
4	Test Execution Results	2
4.1	Overall Results	3
4.2	Executed Test Files	3
5	Validated Functionalities	3
5.1	sb.features	3
5.2	sb.models	3
5.3	sb.smartbugs	4
5.4	sb.cli	4
6	Integration Tests	4
6.1	CLI Integration Test	5
6.2	Smart Selection Integration Test	5
7	Coverage Results	5
8	Manual Validation of the Intelligent Workflow	5
9	Discussion	6
10	Conclusion	6

1 Introduction

This document reports the execution of the tests introduced to validate the new functionalities added to SmartBugs as part of the proposed evolution.

The goal of these tests is to verify the correctness of the newly implemented intelligent workflow, which includes:

- feature extraction through SCsVulLyzer;
- machine learning-based tool selection;
- integration of the intelligent selection phase into the SmartBugs workflow;
- command-line parsing of the new intelligent execution options.

The reported tests focus on the modules introduced or extended to support the new functionality:

- `sb.features`;
- `sb.models`;
- the intelligent orchestration logic inside `sb.smartbugs`;
- the new command-line options handled by `sb.cli`.

2 Test Environment

The tests were executed in the following environment:

- Operating System: Windows
- Python version: 3.11.0
- Test framework: `pytest 9.0.3`
- Additional pytest plugins:
 - `pytest-cov 7.1.0`
 - `pytest-mock 3.15.1`
 - `pytest-timeout 2.4.0`

The execution was performed inside the project virtual environment.

3 Executed Test Suites

The following test suites were executed to validate the new functionalities:

- `tests/unit/test_features.py`
- `tests/unit/test_models.py`
- `tests/unit/test_smartbugs.py`
- `tests/integration/test_cli_integration.py`
- `tests/integration/test_smart_select_integration.py`

The command used to execute the tests was:

```
pytest tests/unit/test_features.py tests/unit/test_models.py \  
tests/unit/test_smartbugs.py tests/integration/test_cli_integration.py \  
tests/integration/test_smart_select_integration.py --cov-fail-under=0
```

4 Test Execution Results

The test execution completed successfully.

4.1 Overall Results

- Total collected tests: 64
- Passed tests: 64
- Failed tests: 0
- Errors: 0
- Skipped tests: 0

Therefore, all executed tests passed successfully.

4.2 Executed Test Files

Test File	Type	Result
tests/unit/test_features.py	Unit	Passed
tests/unit/test_models.py	Unit	Passed
tests/unit/test_smartbugs.py	Unit / Component	Passed
tests/integration/test_cli_integration.py	Integration	Passed
tests/integration/test_smart_select_integration.py	Integration	Passed

Table 1: Executed test suites and results

5 Validated Functionalities

5.1 sb.features

The unit tests for `sb.features` validated the behavior of the feature extraction wrapper introduced in CR.1. In particular, the tests verified:

- correct handling of valid Solidity input files;
- detection of missing contract files;
- detection of missing SCsVulLyzer entry point;
- handling of subprocess execution failures;
- parsing of dictionary-based output;
- parsing of key-value output;
- handling of empty or malformed output.

These tests confirmed that the wrapper correctly isolates the interaction with SCsVulLyzer and returns a structured Python dictionary suitable for the following phases of the intelligent workflow.

5.2 sb.models

The unit tests for `sb.models` validated the machine learning-based tool-selection logic introduced in CR.2. In particular, the tests verified:

- selection of the required subset of 41 features;
- detection of missing required features;
- loading of serialized `.joblib` models;
- handling of invalid or missing model files;
- execution of model predictions;
- handling of invalid prediction outputs;
- aggregation of multiple model predictions;

- removal of duplicate predictions;
- filtering of non-informative predictions such as `nessun_tool`;
- filtering of unsupported tool identifiers.

These tests confirmed that the model-selection module can correctly transform a feature dictionary into a clean list of SmartBugs tool identifiers.

5.3 `sb.smartbugs`

The unit and component tests for `sb.smartbugs` validated both the original orchestration logic and the new intelligent orchestration behavior introduced in CR_3.

The already existing tests continued to validate:

- file collection from input patterns;
- handling of Solidity, bytecode, and runtime bytecode files;
- task generation;
- compiler resolution;
- duplicate handling;
- Docker image loading decisions;
- preservation of the original workflow.

In addition, the new integration-oriented tests validated:

- interaction between intelligent tool selection and the main workflow;
- propagation of the selected tools to the standard SmartBugs pipeline;
- execution order between file collection, intelligent selection, tool loading, and task generation;
- normalization of predicted tool identifiers such as `slither-0.10.4` into SmartBugs identifiers such as `slither`.

These tests confirmed that the intelligent workflow is correctly integrated into the existing orchestration logic.

5.4 `sb.cli`

The new integration test for `sb.cli` validated the command-line parsing of the new intelligent execution options.

In particular, the test verified the correct parsing of:

- `--smart-select`;
- `--feature-extractor`;
- `--model-paths`;
- `--continue-on-errors`;
- `-f / --files`.

This test confirmed that the new options are correctly transferred from the command line into the `Settings` object.

6 Integration Tests

Two dedicated integration tests were introduced for the intelligent workflow.

6.1 CLI Integration Test

The file `tests/integration/test_cli_integration.py` validates the integration between:

- command-line argument parsing in `sb.cli`;
- storage of parsed values inside `sb.settings.Settings`.

This test executes the real CLI parsing logic with simulated command-line arguments and verifies that the resulting settings object contains the expected values.

6.2 Smart Selection Integration Test

The file `tests/integration/test_smart_select_integration.py` validates the integration between:

- `sb.smartbugs`;
- `sb.features`;
- `sb.models`;
- `sb.settings`.

The test uses:

- a temporary Solidity contract;
- a temporary fake SCsVulLyzer entry point;
- temporary serialized `.joblib` models;
- a temporary tools directory compatible with SmartBugs identifiers.

This allows the test to validate the complete intelligent selection phase with real interactions among the new modules, while isolating only the final external execution pipeline based on Docker.

7 Coverage Results

The test execution produced the following relevant coverage values for the modules involved in the new functionality.

Module	Coverage
<code>sb.features.py</code>	88.64%
<code>sb.models.py</code>	100.00%
<code>sb.smartbugs.py</code>	90.40%
<code>sb.settings.py</code>	62.73%
<code>sb.cli.py</code>	excluded from coverage configuration

Table 2: Coverage results for the most relevant modules

The overall project coverage for the executed test command was:

52.50%

This value is influenced by the fact that the execution pipeline modules related to Docker and full analysis execution were not directly exercised in automated form.

8 Manual Validation of the Intelligent Workflow

In addition to automated tests, the intelligent workflow was also validated manually through direct command-line execution.

The following command was executed:

```
python -m sb --smart-select -f samples/0.8.24/ERC20.sol --continue-on-errors
```

During manual validation, the following behavior was observed:

- the input Solidity file was collected correctly;
- the intelligent workflow was activated;
- SCsVulLyzer feature extraction completed successfully;
- the machine learning models were executed;
- the predicted tool list was filtered and normalized;
- the tool `smartcheck` was selected;
- one SmartBugs task was generated;
- execution reached the original Docker-based analysis phase.

This manual validation provided additional evidence that the intelligent workflow behaves correctly in a realistic execution scenario.

9 Discussion

The executed tests confirm that the new functionality introduced in the project is correctly implemented and adequately validated.

In particular:

- CR_1 is supported by dedicated unit tests for feature extraction;
- CR_2 is supported by dedicated unit tests for machine learning-based tool selection;
- CR_3 is supported by integration-oriented tests that validate the interaction between CLI parsing, settings propagation, intelligent selection, and orchestration.

The final Docker-based execution of the selected external tools was not fully automated in the test suite, since it depends on external containers and platform-specific runtime conditions. For this reason, it was validated manually.

10 Conclusion

The new-feature test execution was successful.

All 64 executed tests passed, including both unit tests and integration tests introduced for the intelligent workflow. The results confirm the correctness of the implemented feature extraction wrapper, machine learning-based tool-selection module, command-line integration, and orchestration logic.

Together with the manual validation of the complete intelligent workflow, these results provide sufficient evidence that the proposed extension of SmartBugs behaves as expected.